Module 10 — Reading: Deploying NLP + Knowledge Graph + Retrieval-Augmented Generation as a Multi-Service Stack

> ***Module 9 → Module 10.*** *Module 9 taught you to query a knowledge graph and retrieve facts you can trace back to a node and a relationship. Module 8 taught you to retrieve passages by semantic similarity from a vector store. Module 10 is the first time you ship those pipelines as a* ***service*** *— something with a URL, an OpenAPI document, a clickable frontend, and a container image for every tier. The Python function you have been calling from a notebook becomes an endpoint that anyone with the URL can call.*

What success looks like: By the end of Module 10, you can build a FastAPI service that wraps NLP, knowledge-graph, and retrieval-augmented-generation pipelines as typed REST endpoints, write Dockerfiles for the backend and a Next.js frontend, compose Neo4j, Weaviate, the backend, and the frontend into a one-command four-service stack, build a clickable Next.js client that calls the typed API, instrument the stack with healthchecks and structured errors, and reason about monolithic versus microservice shapes.

> ***Scope note — what "deployment" means in this module.*** *Deployment is a layered idea. This module teaches the* ***unit of shipment*** *— packaging your service as a self-contained, runnable bundle. By the end of the week you can hand someone a* `git clone` *URL and a* `docker compose up -d` *command, and they will have your typed FastAPI backend, your Next.js frontend, and the two stateful backends (Neo4j, Weaviate) running on their own machine: four services healthy, the API documented and browsable at* `/docs`*, and the frontend calling the API at* `http://localhost:3000`*. That bundle is the substrate every hosting platform expects as input. When you eventually want a public URL anyone on the internet can visit, you push your container image to a registry and a managed platform (Render, Fly.io, Railway, Google Cloud Run, and AWS App Runner are common starting points) consumes that image: the platform attaches a TLS / HTTPS (Transport Layer Security / HTTP Secure) certificate, registers the service under a DNS (Domain Name System) hostname, and runs the production-tuned process (multi-worker Uvicorn or gunicorn).* ***Module 11*** *builds the observability layer — counters, latency histograms, automated evaluation harnesses — that runs against this same substrate. The* ***M12 capstone*** *keeps the Compose pattern as its local development surface; teams that want a public URL pick one of those platforms and follow its "Deploy a Dockerfile" guide, which assumes a solid Compose stack as input. The work in this module is the layer every later step is built on top of.*

In This Reading

1. Setup: install before the concept session
2. From notebook to service — what changes
3. FastAPI path operations and the Pydantic boundary
4. Dependency injection and lifespan
5. Wrap shape 1 — stateless function (NLP)
6. Wrap shape 2 — KG (stateful, single backend)
7. Wrap shape 3 — RAG (stateful, two backends + a model)
8. Next.js for Python engineers
9. Containerization with Docker — the Dockerfile recipe (Python)
10. Containerization — the Dockerfile recipe (Node / Next.js multi-stage)
11. docker compose for multi-container stacks
12. Container readiness ordering
13. Service lifecycle — readiness and liveness
14. CORS for the Next.js client
15. Production-discipline error handling
16. Structured logging and request correlation
17. Monolith vs microservice — when each shape pays
18. The carry-forward functions wrapped this module

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. Setup: install before the concept session

Module 10 introduces **two new toolchains** alongside everything you have used since Pre-Work — a Python web framework (FastAPI + Uvicorn) and a JavaScript / TypeScript framework (Next.js, which runs on Node.js). The cumulative install is ~10 minutes on a fresh laptop. Start the Node.js download before you read the rest of this section if your connection is slow.

> ***Hardware-profile sidebar.*** *The Module 10 stack runs the FastAPI backend, the Neo4j knowledge-graph store, the Weaviate vector store, the Next.js frontend, and a held-in-memory* `flan-t5-base` *generator simultaneously.* ***The hardware floor is ≥16 GB RAM.*** *Memory caps in* `docker-compose.yml` *keep Neo4j (heap_max=1G + pagecache=512M) and Weaviate bounded; the generator is loaded once in* `lifespan` *and shared across requests. If your laptop is under 16 GB, ask your instructional team about cloud-VM access before Sunday's lecture.*

1.1 Core install (everyone)

> ***Windows users.*** *The commands below target Bash / Zsh (macOS / Linux / WSL2 / Git Bash). On PowerShell or CMD: activate your venv with* `.venv\Scripts\Activate.ps1` *(PowerShell) or* `.venv\Scripts\activate.bat` *(CMD). Docker Desktop must be running with the WSL2 backend active.*

1.1.1 Verify Docker and pull the M10 images.

```
docker compose version    # must report Compose v2.x
docker ps                 # must succeed without sudo
docker pull python:3.11-slim                          # ~50 MB, ~30 s
docker pull node:20-alpine                            # ~180 MB, ~1 min
docker pull neo4j:5-community                         # already cached from Modul
docker pull semitechnologies/weaviate:1.24.10         # already cached from Modul
```

Neo4j and Weaviate are already cached from Module 9 Week B and Module 8 respectively; if they are not, allow ~3 minutes each on first pull.

1.1.2 Install Node.js LTS (Long-Term Support) — new toolchain.

The Next.js frontend in the Lab and Integration runs on Node.js 20. Install from nodejs.org. Pick the LTS download for your platform.

- **macOS:** download the `.pkg` installer and run it. About ~5 minutes including download.
- **Linux:** the LTS installer or your distro's `nodejs` package. Verify `node --version` reports `v20.x`.
- **Windows:** download the `.msi` installer. The installer adds `node` and `npm` to your `PATH`. Restart your terminal after install.

Verify:

```
node --version     # v20.x.y
npm --version      # 10.x.y (ships with Node.js 20)
```

If `node --version` reports anything older than `v20.0.0`, install the LTS over the top — Next.js 14 requires Node 18.17+ and we standardize on Node 20.

1.1.3 Install the Python packages.

From your course venv:

```
pip install -r requirements.txt
```

For Module 10, the Lab's `api/requirements.txt` ships exact pins (for reproducible CI + Docker builds):

```
fastapi==0.110.0
uvicorn[standard]==0.27.1
pydantic==2.6.1
httpx==0.27.0
spacy==3.7.4
neo4j==5.18.0
weaviate-client==3.26.7
sentence-transformers==2.5.1
transformers==4.39.3
torch==2.2.1
pytest==8.0.2
pytest-asyncio==0.23.5
```

`fastapi`, `uvicorn[standard]`, `pydantic`, and `httpx` are new this module. `neo4j` is carried forward from Module 9 Week B; `spacy`, `weaviate-client`, `sentence-transformers`, `transformers`, and `torch` are carried forward from Module 6 / Module 7 / Module 8 (`torch` is pinned here because `sentence-transformers` and `transformers` need it; pinning prevents a too-new wheel from breaking the Docker build). `pytest-asyncio` is new — the Lab's autograder uses `asyncio_mode = auto` (in `pytest.ini`) so async path-operation tests run without per-test markers.

Verify:

```
python -c "import fastapi, uvicorn, pydantic, httpx; print('ok')"
```

1.1.4 Reuse cached models.

The spaCy `en_core_web_sm` model and the `flan-t5-base` generator are carried forward from Module 6 / Module 8 and should already be cached under `~/.cache/huggingface/` and your venv's `spacy/data/`. No re-download required. If your cache was cleared, install the model by URL — `python -m spacy download` sometimes constructs a malformed URL (`.../download/-en_core_web_sm/-en_core_web_sm.tar.gz` → 404) for certain spaCy/pip combinations:

```
pip install https://github.com/explosion/spacy-models/releases/download/en_core_w
python -c "from transformers import pipeline; pipeline('text2text-generation', mo
```

The `flan-t5-base` cold-download is ~250 MB and ~2 minutes; the spaCy small English model is ~12 MB and a few seconds.

1.1.5 Confirm the Lab and Integration repos can install their `web/` packages.

Module 10 is the first module where the same pip install carries through to the Lab and Integration repos via pip cache (so downstream installs are fast — typically under one minute each). The Lab additionally requires `npm ci` against the pre-generated `package-lock.json` in `web/`. The `npm ci` command is cached after the first run; allow ~1 minute on first install per repo and a few seconds after.

---

2. From notebook to service — what changes

When you call a function from a notebook, the caller and the callee live in the same Python process. The function returns when it returns; if it raises, the notebook cell shows a traceback; if the kernel restarts, the function is reloaded from source. None of this survives outside the notebook.

A **service** is a process that accepts requests over a network protocol and returns responses. The same Python function you have been calling from a notebook can be called by anyone with the URL. This reframes the unit of delivery:

Notebook	Service
One process, one caller	One process, many callers, possibly concurrent
Function signature is the contract	The HTTP (Hypertext Transfer Protocol) request shape is the contract
Failure = traceback printed in your cell	Failure = a status code and a structured error body
State lives in the kernel	State lives in the process — and disappears on restart
Reload via kernel restart	Reload via container restart (or a rolling deploy)
You call the function	The browser calls it — the network is part of the runtime

Three new questions arise:

1. **What is the contract?** The function signature was the contract; now the HTTP request shape is. You declare it with Pydantic so it is enforced at the boundary, not in the function body.
2. **What is the lifecycle?** A connection to Neo4j or Weaviate is expensive to open. In a notebook you may open it once at the top of the file and forget. In a service it must be opened once per process and closed when the process exits.
3. **What is the deployment unit?** A notebook is a `.ipynb` file. A service is a process — and the modern unit of "process I can ship" is a Docker container.

This Module is built around those three questions.

---

3. FastAPI path operations and the Pydantic boundary

FastAPI is a Python web framework that compiles your typed function signatures into an HTTP API (Application Programming Interface) and an auto-generated OpenAPI schema. **OpenAPI** (originally called the Swagger Specification) is a standard JSON-format description of an HTTP API — every endpoint, every input shape, every response shape, every error code — that any tool in any language can read. FastAPI writes yours for you from the Pydantic models and route decorators. The mental model is small: a **path operation** is a function decorated with the HTTP verb and the path it answers on.

```python
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class ExtractRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=5000)


class Entity(BaseModel):
    text: str
    label: str
    start: int
    end: int


class ExtractResponse(BaseModel):
    entities: list[Entity]


@app.post("/extract", response_model=ExtractResponse)
def extract(req: ExtractRequest) -> ExtractResponse:
    # function body: call spaCy, return entities
    ...
```

Three things are happening:

1. **`ExtractRequest` is the input contract.** Pydantic validates `text` at the boundary — empty string, missing field, oversized string, and wrong type are all rejected with HTTP 422 (Unprocessable Entity) and a structured error body before your function runs. You do not write validation code in the function body.
2. **`ExtractResponse` is the output contract.** FastAPI serializes the response and validates it against the model. If your function returns `{"entities": [{"text": ..., "label": ..., "start": ..., "end": ...}]}` the response is sent; if your function returns a Neo4j `Record` (not JSON-serializable) you get a clear server-side error.
3. **`@app.post("/extract")` is the route.** FastAPI compiles `(verb, path, request shape, response shape)` into an OpenAPI document available at `GET /openapi.json` and a browsable Swagger UI at `GET /docs`. The OpenAPI schema is the canonical typed contract — anyone (or any tool) can read it and generate a typed client.

The boundary is the value. Validation happens once, at the request boundary. The function body assumes valid input. The response model documents — and enforces — what the caller sees. There is one contract for both sides.

Flask sidebar (5-line mapping)

If you have used Flask before, the mapping is direct:

Flask	FastAPI
`@app.route("/extract", methods=["POST"])`	`@app.post("/extract")`
`request.json` and manual validation	Pydantic `BaseModel` as the parameter type
`return jsonify({"entities": [...]})`	`return ExtractResponse(entities=[...])`
Manual OpenAPI docs (e.g., Flask-RESTX)	Auto OpenAPI at `/docs`
`@app.errorhandler(ValidationError)`	Pydantic ValidationError → 422 automatically

Flask is a familiar reference shape for engineers who have built a Python web service before; FastAPI is the framework we build with.

---

4. Dependency injection and lifespan

A connection to Neo4j is expensive to open: the Bolt protocol does TLS (Transport Layer Security) negotiation, authentication, and pool setup. The same is true of the Weaviate client and the `flan-t5-base` generator. You want **one of each per process**, not one per request.

FastAPI gives you two tools for this: `lifespan` (a context manager that runs once at startup and once at shutdown) and `Depends()` (a parameter annotation that injects a resource into a path operation).

```python
import os
from contextlib import asynccontextmanager
from fastapi import FastAPI, Depends
from neo4j import GraphDatabase
import weaviate
from transformers import pipeline


@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup — read connection config from environment.
    # The Compose file sets NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD,
    # WEAVIATE_URL on the `api` service. Hardcoding credentials
    # here is a real-world anti-pattern and a guaranteed mismatch
    # with whatever the Compose file actually injects.
    app.state.neo4j_driver = GraphDatabase.driver(
        os.environ["NEO4J_URI"],
        auth=(os.environ["NEO4J_USER"], os.environ["NEO4J_PASSWORD"]),
    )
    app.state.weaviate_client = weaviate.Client(os.environ["WEAVIATE_URL"])
    app.state.generator = pipeline("text2text-generation", model="google/flan-t5-
    # Same sentence-transformers model the seed used at ingest. Loading
    # it once in lifespan avoids per-request reload and keeps the query
    # embedding consistent with the indexed vectors.
    app.state.embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6
    yield
    # shutdown
    app.state.neo4j_driver.close()


app = FastAPI(lifespan=lifespan)


def get_session():
    with app.state.neo4j_driver.session() as session:
        yield session


def get_weaviate():
    return app.state.weaviate_client


def get_generator():
    return app.state.generator


def get_embedder():
    return app.state.embedder


@app.post("/kg/query")
def kg_query(req: KGRequest, session = Depends(get_session)) -> KGResponse:
    # path operation uses the injected session
    ...
```

`lifespan` opens the connections once when the process starts and closes them when it exits. `Depends()` lets a path operation declare what it needs without constructing it. The path operation never sees the Bolt driver itself — only a per-request session yielded from the shared driver.

Why this matters in practice. Loading `flan-t5-base` is a ~30 second operation the first time. If you instantiate the pipeline inside the path operation body, the first request hangs for 30 seconds, subsequent requests are fast, and your test suite intermittently times out as different tests pay the cold-start cost. Loading once in `lifespan` makes the second-request behavior the only behavior callers ever see.

---

5. Wrap shape 1 — stateless function (NLP)

The Module 6 spaCy Named Entity Recognition (NER) pipeline is the simplest wrap: it takes a string, runs `nlp(text).ents`, and returns a list of entities. No per-request resource other than the spaCy pipeline loaded at startup.

```python
from pydantic import BaseModel, Field
import spacy

# loaded once at module import — see note below
nlp = spacy.load("en_core_web_sm")


class Entity(BaseModel):
    text: str
    label: str
    start: int
    end: int


class ExtractRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=5000)


class ExtractResponse(BaseModel):
    entities: list[Entity]


@app.post("/extract", response_model=ExtractResponse)
def extract(req: ExtractRequest) -> ExtractResponse:
    doc = nlp(req.text)
    entities = sorted(
        [Entity(text=ent.text, label=ent.label_, start=ent.start_char, end=ent.en
         for ent in doc.ents],
        key=lambda e: e.start,
    )
    return ExtractResponse(entities=entities)
```

Two points worth noting:

- **The output is ordered.** Pydantic does not order list outputs for you; the *function* must sort before returning. We sort by `start` ascending so the response is deterministic. The Lab autograder checks this.
- **The boundary catches abuse.** `Field(..., min_length=1, max_length=5000)` rejects empty input and oversized payloads at the Pydantic layer. You do not write `if not text: raise HTTPException(...)` in the function body.
- **Why `nlp` loads at module import here, not in `lifespan`.** `spacy.load` is synchronous, cheap (~200 ms), and has no shutdown step. Loading it at module import is the simpler shape and behaves identically for this endpoint. Reach for `lifespan` when (a) startup is slow enough that you want explicit ordering, (b) the resource needs an explicit close, or (c) the resource must be injected per-request via `Depends`. The Lab's KG and RAG endpoints meet all three; this NLP endpoint meets none.

---

6. Wrap shape 2 — KG (stateful, single backend)

The Module 9 Week B deterministic NL→Cypher mapper takes a question, maps it to one of a bounded set of Cypher queries, runs it against Neo4j, and returns the rows. Wrapping it behind an endpoint is mostly contract definition.

```python
from w9b_mapper import deterministic_mapper, UnsupportedQueryError


class KGRequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=500)


class KGResponse(BaseModel):
    cypher: str
    rows: list[dict]
    count: int


@app.post("/kg/query", response_model=KGResponse)
def kg_query(req: KGRequest, session = Depends(get_session)) -> KGResponse:
    try:
        cypher = deterministic_mapper(req.question)
    except UnsupportedQueryError as e:
        raise HTTPException(
            status_code=422,
            detail={"reason": "unsupported_question",
                    "supported_patterns": e.supported_patterns},
        )
    result = session.run(cypher)
    rows = [record.data() for record in result]
    return KGResponse(cypher=cypher, rows=rows, count=len(rows))
```

Two contract decisions are load-bearing:

1. **`UnsupportedQueryError` becomes HTTP 422.** A learner question the mapper does not recognize is a *client* error — the request is well-formed but cannot be answered against the bounded schema. We surface that as 422 with a structured `{"reason": "unsupported_question", "supported_patterns": [...]}` body so the client knows how to retry.
2. **`record.data()` produces a JSON-serializable dict.** Returning a Neo4j `Record` object directly raises `Object of type Record is not JSON serializable` at response time. The wrap is responsible for converting the database driver's native type into something the wire protocol can carry.

---

7. Wrap shape 3 — RAG (stateful, two backends + a model)

The Module 8 RAG (Retrieval-Augmented Generation) pipeline has three stages: **retrieve** chunked documents from Weaviate by semantic similarity; **assemble** a grounded prompt with the retrieved chunks numbered; **generate** an answer with `flan-t5-base` that cites the retrieved chunks by their numbered position. Wrapping it as a typed endpoint adds a fourth stage: **cite** — extract the citation markers from the generated text and refuse to return an answer that does not cite at least one retrieved chunk.

This is the densest wrap of the Module. It composes three subsystems behind one Pydantic contract.

```python
import re
from pydantic import BaseModel, Field


class RAGRequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=500)
    k: int = Field(4, ge=1, le=10)


class Citation(BaseModel):
    chunk_id: int
    score: float


class RAGResponse(BaseModel):
    answer: str
    citations: list[Citation]
    confidence: float


EMPTY_SENTINEL = "I cannot answer this from the available sources"

PROMPT_TEMPLATE = """Answer the question using ONLY the numbered sources below.
Cite each claim with the source number in square brackets, e.g. [1] or [2].
If the sources do not contain the answer, say "{sentinel}".

Sources:
{sources}

Question: {question}
Answer:"""


@app.post("/rag/answer", response_model=RAGResponse)
def rag_answer(
    req: RAGRequest,
    weaviate = Depends(get_weaviate),
    generator = Depends(get_generator),
    embedder = Depends(get_embedder),
) -> RAGResponse:
    # 1. retrieve — encode the question with the same sentence-transformers
    # model the seed used at ingest, then query Weaviate via near_vector.
    # `vectorizer=none` means Weaviate cannot embed the query itself; the
    # near_text shortcut fails at runtime with KeyError: 'data'.
    vector = embedder.encode(req.question).tolist()
    result = (weaviate.query
              .get("Chunk", ["text", "chunk_id"])
              .with_near_vector({"vector": vector})
              .with_additional(["distance"])
              .with_limit(req.k)
              .do())
    chunks = result.get("data", {}).get("Get", {}).get("Chunk", [])

    if not chunks:
        return RAGResponse(answer=EMPTY_SENTINEL, citations=[], confidence=0.0)

    # 2. assemble
    sources = "\n".join(
        f"[{i+1}] {chunk['text']}" for i, chunk in enumerate(chunks)
    )
    prompt = PROMPT_TEMPLATE.format(
        sources=sources, question=req.question, sentinel=EMPTY_SENTINEL,
    )

    # 3. generate
    output = generator(prompt, max_new_tokens=256, do_sample=False)[0]
    answer = output["generated_text"].strip()

    # 4. cite — extract [N]-style markers and refuse if none
    cited_ids = {int(m) - 1 for m in re.findall(r"\[(\d+)\]", answer)
                 if 1 <= int(m) <= len(chunks)}
    if not cited_ids:
        return RAGResponse(answer=EMPTY_SENTINEL, citations=[], confidence=0.0)

    citations = [
        Citation(chunk_id=chunks[i]["chunk_id"],
                 score=1.0 - chunks[i]["_additional"]["distance"])
        for i in sorted(cited_ids)
    ]
    confidence = sum(c.score for c in citations) / len(citations)
    return RAGResponse(answer=answer, citations=citations,
                       confidence=max(0.0, min(1.0, confidence)))
```

Grounding is part of the contract. A RAG endpoint that returns a confidently-worded answer with no sources behind it is worse than no endpoint at all — the caller has no way to tell whether the model knew the answer or hallucinated it. We make grounding explicit by:

1. **Returning the sentinel** `"I cannot answer this from the available sources"` whenever retrieval returns zero chunks.
2. **Returning the sentinel** whenever the generated text has no citation markers (the model declined to ground, or hallucinated past the prompt).
3. **Validating citation IDs** — every `chunk_id` in the response must correspond to a chunk Weaviate actually returned in the top-k. The Lab autograder checks this.

The Lab's `/rag/answer` endpoint is graded against (a) endpoint shape, (b) Weaviate retrieval correctness, (c) grounding (citations present and resolve to retrieved chunks), and (d) the empty-retrieval refusal pattern.

---

8. Next.js for Python engineers

Next.js is a JavaScript / TypeScript framework built on React. It bundles four things you would otherwise stitch together yourself: a file-system router (the file `pages/extract.tsx` answers on the URL `/extract`); a development server with hot reload; a production build pipeline (`next build` + `next start`); and a set of opinionated patterns for server-side rendering and data fetching.

For Module 10 we use a small, deliberate subset:

- **Pages Router** (the older, file-based router — more familiar to learners new to React than the App Router).
- **Three pages** — `pages/extract.tsx`, `pages/kg.tsx`, `pages/rag.tsx` — one per backend endpoint.
- **Client-side `fetch`** for backend calls. No `getServerSideProps`, no Server Components.
- **Hand-written TypeScript interfaces** in `lib/types.ts` mirroring the backend's Pydantic shapes.
- **Environment variable `NEXT_PUBLIC_API_URL`** for the backend base URL. The `NEXT_PUBLIC_` prefix is what makes the variable visible to browser code.

A minimal page looks like this:

```tsx
import { useState } from "react";

interface Entity {
  text: string;
  label: string;
  start: number;
  end: number;
}

interface ExtractResponse {
  entities: Entity[];
}

export default function ExtractPage() {
  const [text, setText] = useState("");
  const [result, setResult] = useState<ExtractResponse | null>(null);
  const [error, setError] = useState<string | null>(null);

  async function submit() {
    setError(null);
    const url = process.env.NEXT_PUBLIC_API_URL + "/extract";
    const res = await fetch(url, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text }),
    });
    if (!res.ok) {
      setError(`Backend returned ${res.status}`);
      return;
    }
    setResult(await res.json());
  }

  return (
    <main>
      <h1>Extract entities</h1>
      <textarea value={text} onChange={e => setText(e.target.value)} />
      <button onClick={submit}>Submit</button>
      {error && <p className="error">{error}</p>}
      {result && (
        <ul>
          {result.entities.map((ent, i) => (
            <li key={i}>{ent.text} — {ent.label}</li>
          ))}
        </ul>
      )}
    </main>
  );
}
```

Why the small surface. Next.js can do server-side rendering with `getServerSideProps`, React Server Components, edge runtimes, and many other production patterns. For a three-page demo client, those buy nothing and cost cognitive load. The Lab and Integration stop where the value stops.

`NEXT_PUBLIC_*` is build-time, not runtime. This catches every Python engineer at least once. The `NEXT_PUBLIC_API_URL` value is *baked into the bundled JavaScript at build time* — when you change `.env.local` and re-run `next dev` or rebuild the container, the new value takes effect; if you change the value after the build with no rebuild, the browser keeps using the old value. The correct value depends on *who is calling*:

- The **browser** runs on the learner's laptop. It can reach `http://localhost:8000` (because port 8000 is published on the host) but it cannot reach `http://api:8000` (the Compose service-name DNS only exists inside the Compose network).
- A **server-side fetch from inside the `web` container** would use `http://api:8000`. We do not use server-side fetches in Module 10.

So in Module 10, `NEXT_PUBLIC_API_URL=http://localhost:8000`. Always. Section 11 returns to this when we walk the Compose file.

---

9. Containerization with Docker — the Dockerfile recipe (Python)

A **Dockerfile** is a recipe that turns a directory of source code into a runnable container image. The recipe is a sequence of layers; each instruction produces a new layer; layers are cached and reused if the inputs have not changed.

The cache-friendly Python recipe is:

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# Copy requirements first so the pip install layer is reused
# when only source code changes (the common case).
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Now copy the rest of the source.
COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Three things are deliberate:

1. **`COPY requirements.txt`** comes *before* `COPY . .`. If you edit `main.py` but not `requirements.txt`, only the final `COPY` layer is rebuilt; the pip install layer (which is the slow one) is reused from cache. Reversing the order rebuilds pip every time you change Python code.
2. **`--no-cache-dir`** keeps pip's cache out of the image. The pip cache is useful on a dev laptop and useless in a production image.
3. **`--host 0.0.0.0`** is required. The default `127.0.0.1` only accepts connections from inside the container — your host machine cannot reach it. Inside containers, always bind to `0.0.0.0`.

`.dockerignore` keeps your build context small. Put a `.dockerignore` next to the `Dockerfile`:

```
.git
.venv
__pycache__
*.pyc
.pytest_cache
.env
data/
tests/
```

Without `.dockerignore`, your `COPY . .` instruction copies your `.venv` (often hundreds of MB) and `data/` (potentially GB) into every layer. The image balloons; CI times out; downloads slow.

Building and running:

```
docker build -t m10-api .
docker run -p 8000:8000 -e NEO4J_URI=bolt://... m10-api
```

---

10. Containerization — the Dockerfile recipe (Node / Next.js multi-stage)

A Next.js production image needs the *built output* (`.next/`) and the *production runtime dependencies* — but not the dev dependencies, the source tree, or the build cache. A naive single-stage build produces a ~700 MB image because it includes everything (full source, `.git`, build artifacts, and devDependencies). A multi-stage build produces a ~250 MB image because the runner stage installs production-only deps and copies only the built artifacts from a throwaway build stage.

```dockerfile
# Stage 1 — builder
FROM node:20-alpine AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .
RUN npm run build

# Stage 2 — runner
FROM node:20-alpine AS runner

WORKDIR /app
ENV NODE_ENV=production

# Install production dependencies only. `npm ci --omit=dev` reads
# package-lock.json and installs the dependencies tree without
# devDependencies. Copying node_modules across from the builder
# would carry devDependencies into the runtime image.
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# Copy the built output and static assets from the builder stage.
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public

EXPOSE 3000

CMD ["npm", "start"]
```

The `node:20-alpine` base is itself small (~180 MB). The builder stage's `node_modules` directory is large (~300 MB for a Next.js project) because it includes devDependencies (`typescript`, `eslint`, `@types/*`, test runners). The runner stage installs only the production subset (~80 MB) and adds the built `.next/` output (~20 MB) and `public/` static assets. The final image weighs in around 250 MB; the disposable builder stage is discarded. Most of the savings come from dropping devDependencies and the source tree — *not* from anything Docker does automatically.

The matching `.dockerignore`:

```
.git
node_modules
.next
.env.local
```

Excluding `node_modules` is critical — without it your local `node_modules` ships into the builder stage and conflicts with `npm ci`, and your image balloons.

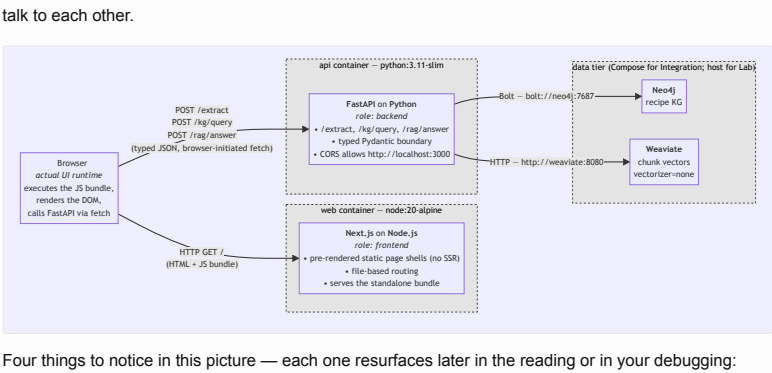
---

11. docker compose for multi-container stacks

`docker compose` is the topology language. A `docker-compose.yml` file declares **services** (containers), **networks** (how they reach each other), and **volumes** (where their state persists). The Module 10 stack has **four services**:

- **`api`** — the FastAPI backend, built from `./api`.
- **`web`** — the Next.js frontend, built from `./web` (multi-stage Node).
- **`neo4j`** — the knowledge-graph store, image `neo4j:5-community`.
- **`weaviate`** — the vector store, image `semitechnologies/weaviate:1.24.10`.

Before the YAML, here is the runtime picture of who talks to whom. The browser independently calls two server-side services — Next.js for the UI assets, FastAPI for the data. The two server containers do not talk to each other.



api container — python:3.11-slim
FastAPI on Python
role: backend
• /extract, /kg/query, /rag/answer
• typed Pydantic boundary
• CORS allows http://localhost:3000
data tier (Compose for Integration; host for Lab)
Neo4j
recipe KG
Weaviate
chunk vectors
vectorizer=none
Bolt — bolt://neo4j:7687
HTTP — http://weaviate:8080
POST /extract
POST /kg/query
POST /rag/answer
(typed JSON, browser-initiated fetch)
Browser
actual UI runtime
executes the JS bundle,
renders the DOM,
calls FastAPI via fetch
HTTP GET /
(HTML + JS bundle)
web container — node:20-alpine
Next.js on Node.js
role: frontend
• pre-rendered static page shells (no SSR)
• file-based routing
• serves the standalone bundle

Four things to notice in this picture — each one resurfaces later in the reading or in your debugging:

- The two arrows leaving the browser are **independent**. The browser does not route its FastAPI calls through Next.js. Next.js's job in Module 10 is producing and serving the UI assets, nothing more — that is what "frontend role" means here, even though Node is a server-side runtime.
- `NEXT_PUBLIC_API_URL=http://localhost:8000` is **host-facing**, not `http://api:8000`, because the browser is not inside the Compose network — it cannot resolve service-name DNS. (Section 8 introduced this; the topology is why.)
- The CORS opt-in on FastAPI (`allow_origins=["http://localhost:3000"]`) is what makes the browser → FastAPI arrow legal. Without it the browser refuses the cross-origin call. Section 14 walks the configuration.
- Neo4j and Weaviate sit **behind** FastAPI — only the backend reaches them, never the browser. Their service-name URLs (`bolt://neo4j:7687`, `http://weaviate:8080`) are internal to the Compose network.

> ***Lab vs. Integration scope.*** *The diagram shows the* ***Integration's four-service Compose topology.*** *The* ***Lab's*** *browser-facing arrows are identical, but Neo4j and Weaviate run* ***outside Compose*** *— started on the host via* `docker run` *— and only* `api` *and* `web` *are in the Lab's* `docker-compose.yml`*. If* `docker compose up` *does not start Neo4j on Lab day, that is by design.*

The Integration's `docker-compose.yml` (abbreviated):

```yaml
services:
  neo4j:
    image: neo4j:5-community
    environment:
      # Single source of truth: .env defines NEO4J_USER + NEO4J_PASSWORD;
      # the neo4j image expects NEO4J_AUTH in `user/password` form.
      NEO4J_AUTH: ${NEO4J_USER}/${NEO4J_PASSWORD}
      NEO4J_dbms_memory_heap_max__size: 1G
      NEO4J_dbms_memory_pagecache_size: 512M
    ports:
      - "7687:7687"
      - "7474:7474"
    volumes:
      - neo4j_data:/data
    healthcheck:
      test: ["CMD-SHELL", "cypher-shell -u neo4j -p ${NEO4J_PASSWORD} 'RETURN 1'
      interval: 10s
      timeout: 5s
      retries: 10

  weaviate:
    image: semitechnologies/weaviate:1.24.10
    environment:
      QUERY_DEFAULTS_LIMIT: 20
      AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED: "true"
      PERSISTENCE_DATA_PATH: /var/lib/weaviate
      DEFAULT_VECTORIZER_MODULE: none
    ports:
      - "8080:8080"
    volumes:
      - weaviate_data:/var/lib/weaviate
    healthcheck:
      test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider", "http://loca
      interval: 10s
      timeout: 5s
      retries: 10

  api:
    # Long-form build: context is the repo root so the api/Dockerfile's
    # `COPY api/...` lines resolve, and `uvicorn api.main:app` runs with
    # the api/ package directory present at /app.
    build:
      context: .
      dockerfile: api/Dockerfile
    environment:
      NEO4J_URI: bolt://neo4j:7687
      NEO4J_USER: neo4j
      NEO4J_PASSWORD: ${NEO4J_PASSWORD}
      WEAVIATE_URL: http://weaviate:8080
      WEB_ORIGIN: http://localhost:3000
    ports:
      - "8000:8000"
    depends_on:
      neo4j:
        condition: service_healthy
      weaviate:
        condition: service_healthy
    healthcheck:
      # `python:3.11-slim` does not ship `curl` — use stdlib `urllib`
      # so the healthcheck has no apt dependency.
      test: ["CMD", "python", "-c", "import urllib.request,sys; sys.exit(0 if url
      interval: 10s
      timeout: 5s
      retries: 10

  web:
    build:
      context: ./web
      args:
        NEXT_PUBLIC_API_URL: http://localhost:8000
    ports:
      - "3000:3000"
    depends_on:
      api:
        condition: service_healthy

volumes:
  neo4j_data:
  weaviate_data:
```

A `.env` file alongside the Compose file supplies the secrets — for Module 10, `NEO4J_USER=neo4j` and `NEO4J_PASSWORD=password` (the demo password; rotate in any real deployment). Compose interpolates `${NEO4J_USER}` and `${NEO4J_PASSWORD}` at parse time, so the neo4j container receives `NEO4J_AUTH=neo4j/password` and the api container receives `NEO4J_USER` + `NEO4J_PASSWORD` separately — matching the `os.environ[...]` reads in the `lifespan` function from Section 4. Keep `.env` in `.gitignore`; ship a `.env.example` instead.

Six details are load-bearing:

1. **Compose service-name DNS.** Inside the Compose network, every service is reachable by its service name. `api` reaches Neo4j at `bolt://neo4j:7687` and Weaviate at `http://weaviate:8080`. The container's own `localhost` does *not* reach the other services.
2. **Browser-facing env is different.** `NEXT_PUBLIC_API_URL=http://localhost:8000` — the browser running on the learner's laptop reaches the API via the host's port mapping. Section 8 covered this; the Compose file is where it surfaces operationally.
3. **`DEFAULT_VECTORIZER_MODULE: none`** tells Weaviate not to vectorize internally. We embed externally with `sentence-transformers` and write pre-computed vectors. Without this, Weaviate tries to vectorize on insert, conflicts with the externally-provided vectors, and silently inserts nothing — your RAG endpoint returns zero retrievals and looks broken.
4. **Named volumes preserve state** across `up` / `down`. `neo4j_data` keeps the graph; `weaviate_data` keeps the vector index. `docker compose down -v` removes the volumes; `docker compose down` (no `-v`) keeps them.
5. **Memory caps on Neo4j.** Without `NEO4J_dbms_memory_heap_max__size=1G` and `NEO4J_dbms_memory_pagecache_size=512M`, Neo4j tries to claim a large fraction of your laptop's RAM and the 16 GB hardware floor is no longer enough for the four-service stack.
6. **The `api` healthcheck uses Python's `urllib`, not `curl`.** The `python:3.11-slim` base image does not ship `curl` in its default apt set; a `curl`-based healthcheck on the `api` service fails silently, the service never reaches `healthy`, and `web`'s `depends_on: { api: { condition: service_healthy } }` blocks the whole stack. Using `python -c "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/readyz').status==200 else 1)"` keeps the healthcheck dependency-free. The `weaviate` and `neo4j` services use `curl` and `cypher-shell` respectively — both ship in their own base images, so those healthchecks work as written.

Bringing the stack up:

```
docker compose up -d
docker compose ps
docker compose logs -f api
docker compose down -v
```

---

12. Container readiness ordering

`depends_on` is the dependency declaration. The default form (a bare list) only waits for the dependency container to *start* — not to be *ready*. A FastAPI process started before Neo4j accepts Bolt connections will crash on its first session request.

The right form is `depends_on: { neo4j: { condition: service_healthy }, weaviate: { condition: service_healthy } }`. Compose waits for both Neo4j and Weaviate to pass their healthchecks before starting `api`. The same chain continues — `web` waits for `api` to be healthy. The full readiness order:

```
neo4j  ──┐
         ├──> api  ──> web
weaviate ─┘
```

Both backends must reach healthy before the API starts. The API must reach healthy before the frontend starts. The Lab's `/healthz` (liveness) and `/readyz` (readiness) endpoints (Section 13) are the healthcheck targets for the `api` service.

Healthchecks are not free — they consume CPU on a polling cadence — but the alternative (race conditions at startup) is worse. Tune the `interval` and `retries` so the first-time learner experience is `docker compose up -d` → 60-90 seconds → everything healthy.

---

13. Service lifecycle — readiness and liveness

There are two questions you ask a service:

- **Liveness — is the process running?** A liveness probe should return 200 as long as the process can answer any HTTP request. If liveness fails, the supervisor (Compose, Kubernetes, the orchestrator) restarts the container.
- **Readiness — can it serve a useful request right now?** A readiness probe should return 200 only if the service's dependencies are reachable. If readiness fails but liveness passes, the supervisor leaves the container running but stops sending it traffic.

For the Lab backend:

```python
@app.get("/healthz")
def healthz():
    return {"status": "ok"}


@app.get("/readyz")
def readyz(
    session = Depends(get_session),
    weaviate = Depends(get_weaviate),
):
    try:
        result = session.run("RETURN 1 AS ok").single()
        assert result["ok"] == 1
    except Exception:
        raise HTTPException(status_code=503, detail={"backend": "neo4j", "status"
    try:
        if not weaviate.is_ready():
            raise HTTPException(status_code=503, detail={"backend": "weaviate", "
    except HTTPException:
        raise
    except Exception:
        raise HTTPException(status_code=503, detail={"backend": "weaviate", "stat
    return {"status": "ready"}
```

`/healthz` must work without backends. A learner who probes Neo4j or Weaviate from `/healthz` and gets a 503 sees the container marked unhealthy and restarted in a loop — even though the FastAPI process is running fine. `/healthz` is process-level; `/readyz` is dependency-level.

---

14. CORS for the Next.js client

By default, browsers refuse cross-origin (Cross-Origin Resource Sharing — CORS) requests. The Next.js frontend served on `http://localhost:3000` cannot call the FastAPI backend on `http://localhost:8000` unless the backend opts in.

FastAPI's CORS middleware:

```python
import os
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.getenv("WEB_ORIGIN", "http://localhost:3000")],
    allow_credentials=True,
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["*"],
)
```

The fingerprint of a missing CORS middleware: the browser console shows "blocked by CORS policy" but `curl` against the same URL works. Learners commonly conclude the backend is broken; the backend is fine — the *browser* is enforcing the CORS rule and the backend has not opted in.

`WEB_ORIGIN` is host-facing (`http://localhost:3000`), not container-facing (not `http://web:3000`) — same reason as `NEXT_PUBLIC_API_URL` in Sections 8 and 11.

---

15. Production-discipline error handling

The error response is part of the API contract. The Module 10 backend uses a small, consistent set of status codes:

Status	Trigger	Response body
422	Pydantic validation failure	`{"detail": [...]}` — FastAPI auto-generates
422	`UnsupportedQueryError` from W9B mapper	`{"reason": "unsupported_question", "supported_patterns": [...]}`
503	Neo4j Bolt connection failure	`{"backend": "neo4j", "status": "unreachable"}`
503	Weaviate connection failure	`{"backend": "weaviate", "status": "unreachable"}`
500	Anything unexpected	`{"detail": "internal_error", "request_id": "..."}`

Two patterns matter:

1. **Define the error shape as a Pydantic model** and use it consistently. The Frontend lead's `lib/types.ts` includes a `BackendError` interface that mirrors the 422/503 shape; the Next.js pages branch on it.
2. **Surface domain errors as 422, not 500.** `UnsupportedQueryError` is a client-side issue (the question is well-formed but cannot be answered against the bounded schema) — 422 with structured detail. Letting it bubble to 500 hides the cause from the caller.

---

16. Structured logging and request correlation

In a notebook you read tracebacks. In a service you read logs — many of them, often concurrent, often partial. The minimum bar is one log line per request, JSON-formatted, with at minimum:

- A **request-id** (generated on entry, propagated through all log lines for the request).
- The **path** and **method**.
- The **status code**.
- The **latency** in milliseconds.

Module 11 builds on this — monitoring counters and the structured-log format are what M11's evaluation hooks read. For Module 10, ship the format; M11 ships the consumers.

A minimal middleware:

```python
import time, uuid, json, logging
from fastapi import Request

logger = logging.getLogger("api")

@app.middleware("http")
async def log_request(request: Request, call_next):
    request_id = str(uuid.uuid4())
    start = time.time()
    response = await call_next(request)
    latency_ms = round((time.time() - start) * 1000, 2)
    logger.info(json.dumps({
        "request_id": request_id,
        "path": request.url.path,
        "method": request.method,
        "status": response.status_code,
        "latency_ms": latency_ms,
    }))
    response.headers["X-Request-Id"] = request_id
    return response
```

---

17. Monolith vs microservice — when each shape pays

The Module 10 Lab backend is **one container with three endpoints** — `/extract`, `/kg/query`, `/rag/answer`. On the application axis that is a *monolith*. The Integration's four-service stack (api + web + neo4j + weaviate) is a *microservice topology* on the deployment axis. Both shapes coexist.

When does each pay?

Question	Monolith backend	Microservice backend
Latency between endpoints	In-process call	Network call
Deploy granularity	One unit	Per-service
Failure isolation	A RAG bug crashes the whole backend	A RAG bug crashes only the RAG service
Resource scaling	Scale the whole backend	Scale the busiest service
Operational cost	One service to monitor	N services + the network between them

A monolith pays when the endpoints share state, are deployed together, and the cost of N service-to-service hops outweighs the failure-isolation benefit. A microservice topology pays when (a) one service needs to scale independently — the RAG service is the obvious candidate because the model inference dominates latency; or (b) one team owns the service and you want their deploys to be independent.

Module 11 extends the stack — split the backend into three microservices, route across them with a coordinator, and instrument the per-service counters. Module 10 ships the application-monolith / stack-microservice shape; Module 11 will demonstrate why and when you would change it.

---

18. The carry-forward functions wrapped this module

This week, you'll wrap the pipelines you built in Modules 6, 8, and 9 Week B behind typed HTTP endpoints.

- **The Module 6 spaCy NER pipeline** — `nlp(text).ents`. Wrapped behind `/extract`.
- **The Module 9 Week B deterministic NL→Cypher mapper** — `deterministic_mapper(question)`. Wrapped behind `/kg/query`.
- **The Module 8 RAG pipeline** — chunked-docs index in Weaviate (`near_text`) → retrieve → assemble cited prompt → `flan-t5-base` generation → extract citation markers. Wrapped behind `/rag/answer`.

Both the W9B mapper and the M8 RAG generator wrapper are **vendored** into the Lab template repo under `api/w9b_mapper/` and `api/m8_rag/`. They ship as DO-NOT-MODIFY modules — your job is to wrap them with typed contracts.

Troubleshooting

- **Port 8000 / 3000 / 7474 / 7687 / 8080 already in use.** `lsof -i :8000` to find the offender; `docker compose down` to release ports the previous run grabbed.
- **Bolt connection refused on first API request.** Neo4j's healthcheck has not yet passed and `depends_on` did not wait. Confirm `depends_on: { neo4j: { condition: service_healthy } }` is set on the `api` service.
- **Weaviate `/v1/.well-known/ready` returns 503 for the first ~30 seconds.** Cold start. The healthcheck `retries: 10` with `interval: 10s` handles it; if learners use bare `depends_on`, the API races Weaviate and loses.
- **Image larger than 1 GB.** Forgot `.dockerignore`. Common culprits: `.venv` (Python image), `node_modules` (Node image), `data/`, `__pycache__`, `.next/` in the source layer.
- **Browser console: "blocked by CORS policy."** CORS middleware missing or `WEB_ORIGIN` does not match the browser's origin. Browser uses `http://localhost:3000`; the backend must allow that exact origin.
- **`NEXT_PUBLIC_API_URL=http://api:8000` and the browser cannot connect.** `api` is a Compose service name; the browser is not inside the Compose network. Use `http://localhost:8000` for `NEXT_PUBLIC_*` values.
- **`lifespan` does not run when `uvicorn --reload` is on.** Known Uvicorn behavior with reload mode. Disable reload when testing `lifespan` initialization.
- **`ModuleNotFoundError` inside the container.** Missing `COPY . .` in the Dockerfile, or the missing module is in a directory excluded by `.dockerignore`.
- **Pydantic v1 idioms (`.dict()`, `parse_obj`, `class Config:`).** Module 10 is Pydantic v2. Use `model_dump()`, `model_validate()`, and `model_config = ConfigDict(...)` instead.
- **`npm ci` fails with `EUSAGE: package-lock.json must be present`.** `npm ci` requires a committed lock file; the Lab repo ships one. If you accidentally ran `npm install` and changed `package-lock.json`, revert it.
- **Next.js build silently bakes `NEXT_PUBLIC_*` from `.env.local`.** Changing the env after build does not change the browser bundle. Rebuild after env changes (`docker compose up --build`).
- **Returning a Neo4j `Record` from a path operation.** `Object of type Record is not JSON serializable`. Convert with `record.data()` before returning.

Key Terms

REST (Representational State Transfer), API (Application Programming Interface), FastAPI, ASGI (Asynchronous Server Gateway Interface), Uvicorn, path operation, Pydantic, `BaseModel`, `Field`, dependency injection, `Depends`, `lifespan`, OpenAPI, Swagger UI, container, image, Dockerfile, base image, layer cache, `.dockerignore`, multi-stage build, `docker compose`, service, named volume, healthcheck, `depends_on`, liveness probe, readiness probe, `/healthz`, `/readyz`, monolith, microservice, CORS (Cross-Origin Resource Sharing), request-id, structured logging, HTTPException, `ServiceUnavailable`, `UnsupportedQueryError`, RAG (Retrieval-Augmented Generation), retrieval, grounding, citation, confidence, Weaviate, vector store, `near_text`, embedding, Next.js, React, Pages Router, `getServerSideProps`, `NEXT_PUBLIC_*`, `next build`, `next start`, hydration, npm (Node Package Manager), `package.json`, `package-lock.json`, `node_modules`.

Summary

Module 10 reframes everything you have built since Module 6 as a *service*. The FastAPI typed boundary (Pydantic in, Pydantic out, OpenAPI documented automatically) is the contract surface. `lifespan` + `Depends()` make connections process-scoped instead of request-scoped. The RAG wrap composes retrieval, prompt assembly, generation, and citation extraction into one endpoint whose contract includes grounding refusal. Next.js gives you a typed browser client with a small surface (three pages, file-based routing, `fetch`). Docker turns each tier into a deployable image; Compose declares the four-service topology with healthchecks, readiness ordering, and named volumes. CORS, structured errors, and a request-id middleware are the production-discipline patterns that turn a working stack into a maintainable one. The Lab is application-monolith / stack-microservice; Module 11 will split the backend further.

What's Next

The **Core Skills Drill** (Sunday night) exercises the typed-API boundary plus a single retrieval call before the Lab adds Neo4j, the full RAG composition, Next.js, and Docker. The **Applied Lab** (Monday–Tuesday) ships the full FastAPI + Next.js Lab. The **Integration Task** (Wednesday–Thursday, **team deliverable**) composes the Lab's two services with containerized Neo4j and Weaviate into the four-service Compose stack. **Module 11** will then orchestrate the stack — monitoring counters, automated evaluation, and (in the stretch) the multi-agent coordinator pattern.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.